

DELTA TRACE: EFFICIENT SIGNED ATTRIBUTION FOR REASONING LANGUAGE MODELS

Anonymous authors

Paper under double-blind review

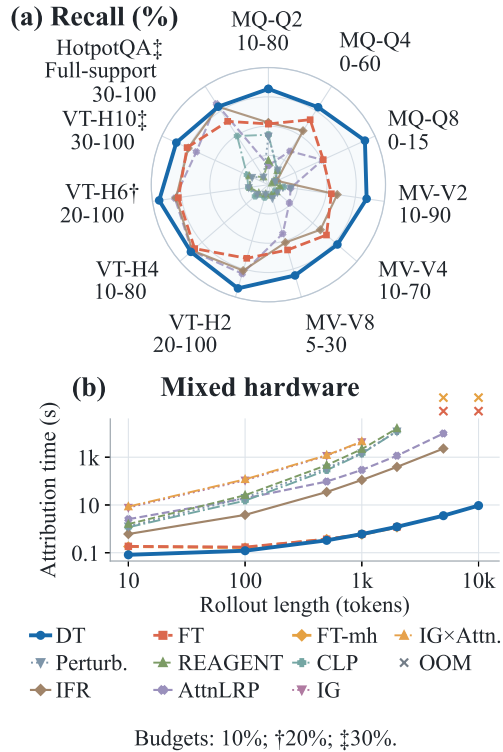
ABSTRACT

We introduce DELTA TRACE, which assigns signed scores to input words to explain a complete reasoning response. It compares the response’s log-probability under the original prompt and a reference that replaces source words with end-of-sequence tokens. Analytic rules convert the observed activation changes into coefficients for one reverse pass, analogous to gradient backpropagation. The token scores sum to the response-score difference. The rules follow content and control together. Attention traces values and their selection, while gated delta memory traces stored content and the keys, queries, and gates controlling retention, writing, and reading. Analytic normalization accounts for competition among attention sources. FlashAttention-style tiling and Flash Linear Attention chunks give sequence-linear auxiliary storage at fixed model dimensions and chunk size. On Qwen3-8B retrieval and reasoning tasks, DELTA TRACE achieves the lowest deletion-faithfulness scores (RISE and MAS) on most tasks and the highest needle-in-a-haystack Recall, exceeding FlashTrace by 5.67–24.33 percentage points. On Qwen3.5-9B, across 13 tasks and 520 examples, DELTA TRACE lowers RISE on 10 tasks and MAS on 12 relative to FlashTrace. Batching with GPU checkpoints increases throughput by 35.3%. The same reference-based formulation motivates policy learning, where exactly averaged action-level return differences recover the advantage signal.

1 INTRODUCTION

Input attribution assigns scores to the words in a prompt to explain a model’s response. A reasoning response depends on both the information available and the computations that select and combine it (Ferrando & Voita, 2024). Figure 1 shows a passage containing a playwright, William Shakespeare, and a composer, William Walton. The names supply candidate answers, and the phrase “the author of the play” specifies which role to use. An explanation should follow both the supplied content and the control that directs its use. It should also distinguish words that support the response from words that oppose it.

We ask how replacing selected prompt tokens would change the model’s score for a chosen response span. The reference replaces these source tokens with end-of-sequence (EOS) tokens at the same positions. Both executions score the same target span using the fixed response prefix. The target can be an answer span or the complete reasoning and answer. We assign each source token a signed share of the re-



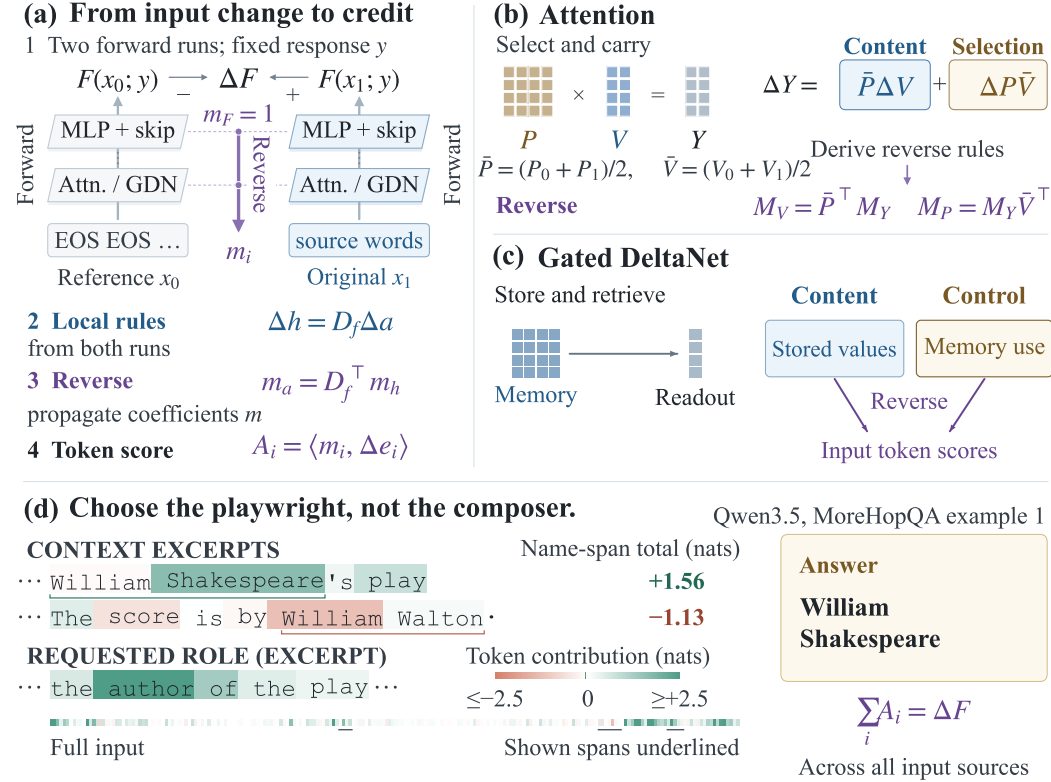


Figure 1: **DeltaTrace follows content and control.** (a) Gray arrows show the original and reference forward runs. The purple reverse pass propagates coefficients backward to compute token scores that sum to their score difference ΔF . (b) Attention assigns credit through values and selection. Bars denote averages of the two runs. (c) GDN assigns credit through stored values and the operations controlling their use. Both paths continue to input token scores. (d) Qwen3.5 assigns positive scores to the playwright’s name and negative scores to the composer’s. Colors saturate at ± 2.5 nats, and annotations sum the marked names. Ellipses omit text. The target includes the complete stored response and EOS.

sulting log-probability difference, so the contributions add across words or passages.

Reference-based methods already assign prediction differences to inputs through integrated gradients or propagated activation differences (Sundararajan et al., 2017; Shrikumar et al., 2017). Relevance propagation and representation decomposition trace how information moves through attention and recurrent states (Ali et al., 2022; Achibat et al., 2024; Rezaei Jafari et al., 2024; Modarressi et al., 2023; Pitorro & Treviso, 2025). FlashTrace follows source importance through multi-token responses (Pan et al., 2026). Our goal is to combine a signed score for a complete response with explicit attribution through the operations that select, store, and retrieve its evidence.

The difficulty is that content and control change together. Changing a token can alter both its value vector and the attention weights that select it. Those weights compete because their sum is one. In gated memory, a new value changes a state assembled from earlier tokens, while keys, queries, and gates determine what is written, retained, and read (Yang et al., 2025). Tracing only the transported content leaves these selection and memory decisions outside the explanation. Tracing both requires local rules that account for their joint changes and can be composed across the whole response.

We introduce DELTATRACE, which turns two forward executions into signed input contributions through one reverse pass. The forward runs supply the original and reference activations. Analytic rules use these pairs to replace each local Jacobian with a finite change map. The reverse pass

applies the transposed maps to coefficients, starting with 1 at the response score. At the input, each coefficient vector is dotted with its embedding difference to obtain a token’s contribution. Thus the forward runs provide the changes, while the reverse pass supplies their multipliers. Attention rules follow content and selection, while memory rules follow retention, writing, and reading. The resulting token contributions sum to the response-score difference.

The computation follows the same structure as accelerated attention. FlashAttention-style tiles process small blocks of token interactions (Dao et al., 2022), and Flash Linear Attention (FLA) processes memory in consecutive chunks (Yang & Zhang, 2024). Accumulating coefficients within these blocks gives auxiliary storage linear in sequence length at fixed model dimensions and chunk size.

On Qwen3-8B retrieval and reasoning tasks, DELTATRACE achieves the lowest deletion-faithfulness scores on most tasks and the highest evidence recovery across the six needle-in-a-haystack tasks (Section 4). Qwen3.5-9B measurements evaluate finite reverse propagation through attention and gated memory. Appendix F develops a policy-learning extension in which averaging action-level return differences over a policy reference recovers the advantage, the return above that policy’s expected return (Sutton et al., 1999; Foerster et al., 2018).

2 DELTATRACE

2.1 THE COUNTERFACTUAL QUESTION AND ATTRIBUTION TARGET

We explain how replacing selected prompt tokens changes the score of a fixed target span. The inputs are a trained language model with fixed parameters θ , a prompt x containing the context and question, and a stored response $y = (y_1, \dots, y_L)$ of L tokens. Let $\mathcal{T}$ be the positions of the target span within y . Both executions score these positions using the same preceding response tokens. The scalar target score is

$$F(x; y) = \sum_{t \in \mathcal{T}} \log p_{\theta}(y_t \mid x, y_{<t}), \quad (1)$$

where p_{θ} is the model’s next-token probability and $y_{<t}$ is the response prefix before position t . A higher score means the model assigns greater probability to the target span given its preceding tokens. For full-response attribution, y includes the reasoning, answer, and end-of-sequence (EOS) token, and $\mathcal{T} = \{1, \dots, L\}$. The response tokens and target positions stay fixed, while their hidden representations can change as they read the different prompts.

We call the prompt positions chosen for explanation the source positions. They can include context words and question words. Let x_1 be the original prompt and x_0 its reference. We construct x_0 by replacing the selected source tokens with EOS at the same positions. The remaining prompt tokens stay fixed. The two forward executions give $\Delta F = F(x_1; y) - F(x_0; y)$. A positive ΔF means these replacements lower the target score. We assign each source token a signed share A_i of this joint score difference, with the contributions adding to ΔF . Positive and negative token scores express support and opposition under this reference, as illustrated in Figure 1.

We use paired subscripts 0 and 1 for reference and original activations, and write $\Delta U = U_1 - U_0$ for their difference. For source token i , $e_{i,0}$ and $e_{i,1}$ are its input embedding vectors, so $\Delta e_i = e_{i,1} - e_{i,0}$. Tokens held fixed have zero embedding difference. The reverse computation below converts these vector differences into the scalar scores A_i .

2.2 TWO FORWARD RUNS AND ONE REVERSE PASS

Figure 1(a) shows four steps. We collect paired activations in two forward executions, obtain local rules from those activations, propagate coefficients backward, and form token scores at the input. The forward executions supply the actual changes, and the reverse pass computes the multipliers that turn those changes into contributions to ΔF .

For an operation $h = f(a)$, a collects its input activations and h is its output. Let J_f be the matrix of local derivatives at one execution. A Jacobian–vector product (JVP) maps an infinitesimal input

change da to an output change dh . A vector–Jacobian product (VJP) maps an incoming gradient g_h back to the input gradient g_a .¹ Table 1 shows the corresponding finite relations.

Table 1: **From gradients to finite reverse propagation.** Paired activations define the finite map D_f that replaces J_f . Changes appear in the left column, and coefficients propagate backward in the right column.

Method	Change relation (JVP form)	Backward coefficients (VJP form)
Gradient backpropagation	$dh = J_f da$	$g_a = J_f^\top g_h$
DELTA TRACE	$\Delta h = D_f \Delta a$	$m_a = D_f^\top m_h$

An analytic rule constructs D_f from the two executions so that $\Delta h = D_f \Delta a$. This identity specifies how the input changes account for the output change. For a scalar nonlinearity, $D_f = \Delta h / \Delta a$ is the secant slope joining the observed points, replacing the tangent slope $f'(a_1)$. At coincident inputs, it becomes the derivative. For operations with multiple inputs, the rules below specify their joint allocation. The algorithm obtains its activation differences from the two forward executions and evaluates the transpose products during the reverse pass.

The backward vector m_h supplies a multiplier for each component of Δh . Their dot product $\langle m_h, \Delta h \rangle$ is the contribution assigned at that output. Substituting the local change relation gives $\langle m_h, D_f \Delta a \rangle = \langle D_f^\top m_h, \Delta a \rangle$. Reading off the input multipliers yields the reverse rule $m_a = D_f^\top m_h$.

Starting with $\Delta F = 1 \cdot \Delta F$, the reverse pass repeatedly rewrites weighted output changes as weighted input changes. At a residual addition, the coefficient passes to each summand. Coefficients returning from multiple consumers add. This reaches the input embeddings and gives

$$A_i = m_i^\top \Delta e_i, \quad \sum_i A_i = \Delta F. \quad (2)$$

Each component of m_i multiplies the corresponding embedding change, and summing those products gives the token score A_i . A fixed token has $\Delta e_i = 0$ and contributes zero. The scores use natural-log units (nats) and add into name, sentence, or passage scores. Appendix A.1 derives the conservation identity.

2.3 WHAT ATTENTION CARRIES AND SELECTS

An attention head lets each position read its own representation and those of earlier positions. Its query describes what to retrieve, each source key describes how to match that source, and each value carries the source’s content. Query–key similarity scores are normalized by softmax into attention weights. Let P_{ij} be the weight with which position i reads source j , and let the rows of V contain the source value vectors. The output $Y = PV$ is their weighted sum.

Changing a source can change both its value and the weight assigned to it. With endpoint means $\bar{P} = (P_0 + P_1)/2$ and $\bar{V} = (V_0 + V_1)/2$, we split these changes symmetrically as

$$\Delta Y = \underbrace{\bar{P} \Delta V}_{\text{content change}} + \underbrace{\Delta P \bar{V}}_{\text{selection change}}. \quad (3)$$

The first term carries changed values using the mean attention weights. The second applies the selection change to the mean values. Their sum is exactly the observed output difference, with the interaction $\Delta P \Delta V$ shared equally between the two terms. In the running example, the name supplies content, while the requested role can change which person’s information is selected.

The two forward executions supply the endpoints for $\bar{P}$ and $\bar{V}$. Given an incoming coefficient matrix M_Y , Equation 3 rewrites $\langle M_Y, \Delta Y \rangle$ as $\langle \bar{P}^\top M_Y, \Delta V \rangle + \langle M_Y \bar{V}^\top, \Delta P \rangle$. The coefficients of the two input changes are therefore

$$M_V = \bar{P}^\top M_Y, \quad M_P = M_Y \bar{V}^\top. \quad (4)$$

¹PyTorch documentation for `torch.func.jvp` and `torch.func.vjp`.

These updates use $\bar{P}$, $\bar{V}$, and the incoming M_Y directly. The identity for ΔY explains their allocation. The quantities returned backward are M_V and M_P . Both coefficient paths continue to the input representations that produced V and P .

Selection is relative because the attention weights in a row sum to one, so increasing one source’s share reduces the total share of the others. The finite softmax rule carries this competition backward by subtracting a weighted row average from the incoming coefficients. Its weights depend on the probabilities in both executions. A source therefore receives selection credit relative to the alternatives that the same query could read. The coefficients then pass through the query–key scores to their input representations. The same construction handles the vocabulary normalization used to score each response token. Appendix A.2 gives the rule and its derivation.

2.4 WHAT MEMORY KEEPS AND UPDATES

Qwen3.5 interleaves attention with Gated DeltaNet memory (Yang et al., 2025). A memory layer carries a state matrix S_{t-1} from one token position to the next. This matrix stores associations between keys and values. At position t , the model supplies a key k_t , a value v_t , a read query q_t , a retention gate α_t , and a write gate β_t . The two gates are scalars controlling how much information is kept and written.

The update first retains $T_t = \alpha_t S_{t-1}$. Reading this retained state at key k_t gives $T_t^\top k_t$, the value already stored for that key. The correction $u_t = \beta_t(v_t - T_t^\top k_t)$ writes the difference between the incoming and stored values. The new state and the output read by the query are

$$S_t = \underbrace{T_t}_{\text{retained content}} + \underbrace{k_t u_t^\top}_{\text{write correction}}, \quad o_t = S_t^\top q_t. \quad (5)$$

Here o_t is the memory readout at position t . It is normalized to n_t , multiplied elementwise by an activated output-gate vector s_t , and linearly projected to form the layer output. This sequence lets early information survive later updates and reach the response.

Reverse coefficients first split between normalized content and the output gate. The memory readout then sends coefficients to the state and query. Reversing the updates carries them to earlier states, values, keys, and retention and write gates. The coefficients continue to the input embeddings, where Equation 2 produces token scores. A word can therefore receive credit for providing information and for changing how that information is stored or retrieved.

At each GDN layer, we apply the finite reverse recurrence twice to the same readout coefficients. The second propagation exchanges the roles of the original and reference activations. Each propagation covers the complete memory recurrence and returns coefficients for values, keys, queries, and retention and write gates. We average these returned coefficients within the layer, then continue backward. The output product $n_t \odot s_t$ also uses symmetric allocation between content and gate changes. Appendix A.4 gives the finite recurrence and output-gating rules.

The remaining network operations complete the same reverse pass in both model families. Linear layers use their weight matrices, additions pass coefficients to their inputs, and nonlinearities and normalizations use finite rules. Appendix A.3 lists the choices for each operator, including symmetric interaction allocation for attention values, query–key products, and feed-forward gating.

3 EFFICIENT COMPUTATION

The reverse pass needs coefficients for input activations at each layer. Keeping an attention matrix for every layer would require storage for all token pairs. DELTATRACE instead computes local transpose products in small blocks and accumulates their results directly into the input coefficients. The attention and memory paths use the following two forms of this idea.

Attention in tiles. A tile is a small block of query and source positions. Following FlashAttention (Dao et al., 2022), DELTATRACE reconstructs the two executions’ attention probabilities for one tile, uses them to update query, key, and value coefficients, and reuses the workspace for the next tile. The softmax rule needs a weighted average over the whole row. One sweep collects those

row statistics. Subsequent sweeps use them when propagating through each tile. For N input-plus-response positions and head dimension d , the stored vectors and row statistics occupy $O(Nd)$ auxiliary space per head. Dense attention requires $O(N^2d)$ arithmetic.

Memory in chunks. A chunk is a consecutive group of tokens. The memory state at a chunk boundary summarizes what earlier tokens passed forward. In reverse, a coefficient for that state carries credit back to the preceding chunk. Within each chunk, matrix products distribute the coefficients across its reads, writes, and gates. This organization uses Flash Linear Attention (FLA) (Yang & Zhang, 2024). At fixed state dimensions and chunk size, the stored token coefficients and boundary states grow linearly with sequence length.

The complete attribution call includes two forward executions and one reverse traversal. It re-computes intermediate activations one layer at a time during the reverse traversal, so each layer’s workspace can be reused for the next. Appendix B details the storage accounting and accelerated operations. Section 4 measures the complete call’s time and memory.

4 EVALUATION

We evaluate whether token scores recover the evidence used in a response, predict the effect of removing that evidence, and can be computed efficiently.

Tasks and methods. We use Qwen3-8B and the 1,243 examples released with FlashTrace (Pan et al., 2026). These comprise ten RULER tasks with 100 examples each, HotpotQA with 48, MATH with 100, and MoreHopQA with 95.² Needle-in-a-haystack (NIAH) tasks retrieve information among distractors. Multi-query tasks (MQ-Q2/Q4/Q8) ask for 2, 4, or 8 keys, while multi-value tasks (MV-V2/V4/V8) ask for 2, 4, or 8 values of one key. Variable Tracking (VT) follows variable assignments. Here H is the number of binding hops and C the number of parallel chains. HotpotQA and MoreHopQA require combining facts, while MATH contains mathematical reasoning problems. The complete stored reasoning and answer plus EOS is scored for faithfulness. We compare DELTA-TRACE (DT) with FlashTrace (FT), Perturbation, REAGENT, Context Length Probing (CLP), Information Flow Routes (IFR), and Attention-aware Layer-wise Relevance Propagation (AttnLRP). DT and FT use paired re-evaluation. VT/HotpotQA recovery compares all seven methods.

Evidence recovery. Recall is the fraction of annotated evidence recovered among the highest-ranked sources. A token budget limits how much input the ranking may select. NIAH selects the top 10% of eligible input tokens ranked by positive-part attribution scores. Eligible tokens are the source positions marked for evaluation in the released examples. VT measures token Recall in the passage body, with budgets of 10%, 10%, 20%, and 30% for H2, H4, H6, and H10, and attributes the answer tokens. HotpotQA reports Full-support Recall@30%. An example scores one if every official supporting fact is selected, and zero otherwise. It attributes the full response, ranks native sentences by the sum of their positive token contributions, and selects the longest ranking prefix within 30% of all body tokens. FT uses three recursive tracing steps (K3) for recovery. Table 4 reports these values.

Deletion faithfulness. The released deletion test masks high-ranked source tokens with a baseline token in 20 steps, rescoring the same complete response after each step. The score is normalized between the full-input and fully-deleted endpoints. A good ranking causes an early score drop because it removes evidence the model uses. The RISE deletion score summarizes the area under this curve. Magnitude Aligned Scoring (MAS) also measures how the removed attribution mass aligns with the score change (Pan et al., 2026). Lower values are better for both metrics. DT uses signed scores for RISE and their positive part for MAS. Appendix C specifies the ranking and normalization.

Attribution quality. With symmetric mean-difference propagation, DT has the lowest RISE on 10 of 13 tasks and the lowest MAS on 11 (Tables 2 and 3). The largest absolute RISE reduction against FT is on MATH, from 0.3487 to 0.2321. DT achieves the highest Recall on all six NIAH

²<https://github.com/wbopan/flashtrace/releases/tag/table1-data-v1>

Table 2: RISE $\downarrow$ on the complete released Qwen3-8B tasks. FT is FlashTrace (Pan et al., 2026) and DT is DELTATRACE. DT uses symmetric mean-difference propagation and signed ranking. FT uses the paired re-evaluation. n is the released task size. Bold marks the lowest observed mean across all seven methods.

Task	n	Perturbation	REAGENT	CLP	IFR	AttnLRP	FT	DT
MQ-Q2	100	0.0945	0.1174	0.0977	0.0747	0.1955	0.0682	0.0723
MQ-Q4	100	0.2389	0.2599	0.2534	0.1147	0.2628	0.1134	0.1448
MQ-Q8	100	0.4986	0.4865	0.5096	0.3709	0.3766	0.3519	0.4286
MV-V2	100	0.1343	0.1876	0.1559	0.0691	0.1403	0.0686	0.0597
MV-V4	100	0.1862	0.2107	0.2166	0.0726	0.1934	0.0704	0.0587
MV-V8	100	0.3514	0.3685	0.3929	0.2051	0.2852	0.1828	0.1159
VT-H2-C3	100	0.3844	0.4378	0.3742	0.1609	0.3191	0.1317	0.1003
VT-H4-C1	100	0.3538	0.3971	0.3277	0.1018	0.3239	0.1102	0.1013
VT-H6-C1	100	0.4584	0.4860	0.4225	0.1245	0.3378	0.1224	0.1109
VT-H10-C1	100	0.4663	0.4953	0.4510	0.1526	0.3573	0.1427	0.1403
HotpotQA	48	0.1333	0.1445	0.1013	0.0745	0.1548	0.0331	0.0275
MATH	100	0.3802	0.3878	0.3616	0.3537	0.3684	0.3487	0.2321
MoreHopQA	95	0.2489	0.2652	0.2487	0.1455	0.2857	0.1281	0.0703

Table 3: MAS $\downarrow$ on the same released tasks and seven methods as Table 2. DT uses its positive source contributions. Bold marks the lowest observed mean.

Task	n	Perturbation	REAGENT	CLP	IFR	AttnLRP	FT	DT
MQ-Q2	100	0.1442	0.1966	0.1663	0.1397	0.3261	0.1628	0.1176
MQ-Q4	100	0.3270	0.3574	0.3203	0.1770	0.4509	0.1937	0.2011
MQ-Q8	100	0.7093	0.6942	0.6566	0.4600	0.6020	0.4271	0.6047
MV-V2	100	0.1870	0.2756	0.2159	0.1343	0.2290	0.1569	0.1046
MV-V4	100	0.2436	0.2905	0.2804	0.1416	0.3248	0.1605	0.1058
MV-V8	100	0.4580	0.4938	0.5114	0.2746	0.4752	0.2477	0.1717
VT-H2-C3	100	0.5513	0.6680	0.4900	0.2309	0.5214	0.1873	0.1486
VT-H4-C1	100	0.5168	0.6031	0.4199	0.1479	0.5475	0.1656	0.1406
VT-H6-C1	100	0.6837	0.7453	0.5648	0.1727	0.5723	0.1787	0.1537
VT-H10-C1	100	0.7013	0.7405	0.5967	0.2011	0.5915	0.1983	0.1842
HotpotQA	48	0.2203	0.2351	0.1895	0.1655	0.2493	0.1276	0.1002
MATH	100	0.5202	0.5358	0.4907	0.4897	0.5244	0.4461	0.3323
MoreHopQA	95	0.3470	0.3679	0.3478	0.2277	0.4579	0.2049	0.1546

tasks and all four VT tasks. At the stated VT budgets, its macro Recall is 87.75%, compared with 76.57% for FT K3. On HotpotQA, DT reaches 85.42% Full-support Recall, compared with 75.00% for FT K3 (Table 4).

Qwen3.5-9B results. Table 5 reports symmetric GDN results on 13 tasks comprising 520 examples, including all 400 VT and 48 HotpotQA examples. DT has lower RISE on 10 tasks and lower MAS on 12. Recall is higher on all six NIAH and all four VT tasks. On HotpotQA, DT reaches 95.83% Full-support Recall, compared with 58.33% for FT K3, and lowers both RISE and MAS.

Attribution time and memory. The first-page timing panel varies response length on Qwen3-8B. DT, FT, and multi-hop FT (FT-mh) are measured on one MetaX C550. The other curves show the published timing references. Crosses mark out-of-memory runs. On the 16-example Qwen3-8B benchmark, the sum of per-example median attribution times is 11.40 seconds for DT and 12.16 seconds for one-hop FT. Peak allocated memory is 18.76 and 21.49 GB. On Qwen3.5-9B, processing the 16 examples in batches of two with GPU checkpoints reduces time from 16.68 seconds for serial execution to 12.33 seconds, a 35.3% throughput increase. Peak memory is 21.64 and 23.88 GB, respectively. Times cover complete attribution calls. Appendix D gives the measurement protocol.

Table 4: Evidence recovery (% , higher is better) at the listed token budgets. NIAH/VT report token Recall. HotpotQA reports Full-support Recall@30%, the fraction of examples for which every official supporting fact is selected. Sentence scores sum positive token contributions. The budget charges all body tokens. VT macro averages its four rows. FT uses K3 for recovery. Bold marks the highest observed mean.

Task	<i>n</i>	Budget	Perturbation	REAGENT	CLP	IFR	AttnLRP	FT	DT
MQ-Q2	100	10%	39.05	24.35	39.85	47.10	21.49	46.32	67.24
MQ-Q4	100	10%	9.04	8.49	8.58	32.84	20.42	39.58	47.16
MQ-Q8	100	10%	0.97	0.47	0.84	1.19	7.63	7.72	13.59
MV-V2	100	10%	25.48	18.02	20.67	57.50	25.44	53.46	77.79
MV-V4	100	10%	16.05	15.60	14.61	45.24	24.25	49.37	56.79
MV-V8	100	10%	8.02	7.37	7.29	17.87	15.88	19.52	25.19
VT-H2-C3	100	10%	26.69	26.66	29.10	81.16	83.37	72.41	93.81
VT-H4-C1	100	10%	18.86	17.58	20.53	70.09	70.54	68.32	71.31
VT-H6-C1	100	20%	33.57	32.47	33.85	83.74	84.57	82.15	95.37
VT-H10-C1	100	30%	40.70	42.48	44.04	82.66	77.33	83.41	90.51
VT macro	400	mixed	29.96	29.80	31.88	79.41	78.95	76.57	87.75
HotpotQA	48	30%	35.42	35.42	64.58	85.42	87.50	75.00	85.42

Table 5: Qwen3.5-9B quality with symmetric GDN propagation. NIAH uses ten new key/value assignments per task on fixed templates. MATH and MoreHopQA each use six fixed released examples. VT and HotpotQA use all released examples. Recovery is token Recall for NIAH/VT and Full-support Recall@30% for HotpotQA, as defined in Table 4. FT denotes FlashTrace, with K3 for Recall and K1 for RISE/MAS. DT uses signed RISE and positive-part MAS. Bold marks the better mean. Dashes indicate undefined recovery.

Task	<i>n</i>	Budget	Recall (%) $\uparrow$		RISE $\downarrow$		MAS $\downarrow$	
			FT	DT	FT	DT	FT	DT
MQ-Q2	10	10%	62.87	80.59	0.1014	0.1012	0.2535	0.1325
MQ-Q4	10	10%	36.86	52.28	0.1504	0.1585	0.2675	0.2185
MQ-Q8	10	10%	11.97	15.31	0.3393	0.3544	0.4017	0.5044
MV-V2	10	10%	64.30	87.11	0.0971	0.0798	0.2452	0.1095
MV-V4	10	10%	50.41	65.34	0.1068	0.1029	0.2471	0.1390
MV-V8	10	10%	24.27	29.50	0.2357	0.2127	0.3178	0.2965
MATH	6	—	—	—	0.3734	0.2662	0.4801	0.3460
MoreHopQA	6	—	—	—	0.1920	0.1327	0.2763	0.1845
VT H2-C3	100	10%	67.80	94.20	0.1514	0.1095	0.2224	0.1295
VT H4-C1	100	10%	69.07	71.89	0.1161	0.1013	0.1888	0.1343
VT H6-C1	100	20%	83.06	97.35	0.1300	0.1154	0.2051	0.1504
VT H10-C1	100	30%	81.90	94.23	0.1537	0.1561	0.2307	0.2040
HotpotQA	48	30%	58.33	95.83	0.2154	0.1470	0.3125	0.2367

Reading the token scores. Figures 3 and 4 show retrieval and multi-hop examples with each model’s own scores. Teal marks positive contributions and coral negative contributions to the complete response score. The retrieval example links source keys to their numbers and the query naming them. The multi-hop example follows a voice actor to a film and its director, whose surname determines the answer. Each deletion curve compares DT with one-hop FT on the same model and response. These views connect token-level scores to the evidence selected and the effect of deleting it.

5 RELATED WORK

Reference-based attribution. Integrated Gradients integrates input sensitivities between a baseline and the input, while DeepLIFT propagates activation differences into source contributions (Sundararajan et al., 2017; Shrikumar et al., 2017). Conservative Propagation and AttnLRP develop rel-

evance rules for transformers (Ali et al., 2022; Achitibat et al., 2024). DT uses a reference difference for the log-probability of a complete response and supplies finite rules for attention selection and gated memory. The local rules determine how input tokens share that score difference.

Tracing source information. ALTI aggregates token interactions across layers, and DecompX propagates decomposed representations to the prediction head (Ferrando et al., 2022; Modarressi et al., 2023). Information Flow Routes traces influential computations, while FlashTrace aggregates response spans and recursively follows their source importance (Ferrando & Voita, 2024; Pan et al., 2026). DT combines the response positions in one scalar target and follows both the transported values and the controls selecting them.

Recurrent models. MambaLRP conserves relevance through selective state-space computation, and LaTIM decomposes token interactions in Mamba models (Rezaei Jafari et al., 2024; Pitorro & Treviso, 2025). DT follows the retained state, write correction, and query read of Gated Delta Networks (Yang et al., 2025). Its memory and attention paths both track content and control changes, with computation organized through FlashAttention tiles and FLA chunks (Dao et al., 2022; Yang & Zhang, 2024).

6 CONCLUSION

DT assigns signed token contributions to the change in a complete response’s log-probability between reference and original inputs. Its reverse pass composes finite changes through the model, following both the evidence carried by values and the attention or memory operations governing its use. The resulting token scores sum to the response-score difference. Tiled attention and chunked memory make the auxiliary storage linear in sequence length at fixed model dimensions and chunk size. The experiments measure evidence recovery, deletion faithfulness, and attribution cost on attention and hybrid models. The same reference-based construction motivates the policy-learning extension in Appendix F.

AI USE STATEMENT

AI tools assisted with method development, derivations, coding, experiments, literature review, and manuscript preparation.

REFERENCES

- Reduan Achitibat, Sayed Mohammad Vakilzadeh Hatefi, Maximilian Dreyer, Aakriti Jain, Thomas Wiegand, Sebastian Lapuschkin, and Wojciech Samek. AttnLRP: Attention-aware layer-wise relevance propagation for transformers. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pp. 135–168. PMLR, 2024. URL <https://proceedings.mlr.press/v235/achitibat24a.html>.
- Ameen Ali, Thomas Schnake, Oliver Eberle, Grégoire Montavon, Klaus-Robert Müller, and Lior Wolf. XAI for transformers: Better explanations through conservative propagation. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162, pp. 435–451. PMLR, 2022. URL <https://proceedings.mlr.press/v162/ali22a.html>.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *arXiv preprint arXiv:2205.14135*, 2022. URL <https://arxiv.org/abs/2205.14135>.
- Javier Ferrando and Elena Voita. Information flow routes: Automatically interpreting language models at scale. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 17432–17445. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.emnlp-main.965. URL <https://aclanthology.org/2024.emnlp-main.965/>.

- Javier Ferrando, Gerard I. Gállego, and Marta R. Costa-jussà. Measuring the mixing of contextual information in the transformer. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 8698–8714. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.emnlp-main.595. URL <https://aclanthology.org/2022.emnlp-main.595/>.
- Jakob Foerster, Gregory Farquhar, Triantafyllos Afouras, Nantas Nardelli, and Shimon Whiteson. Counterfactual multi-agent policy gradients. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, pp. 2974–2982, 2018. doi: 10.1609/aaai.v32i1.11794. URL <https://ojs.aaai.org/index.php/AAAI/article/view/11794>.
- Ali Modarressi, Mohsen Fayyaz, Ehsan Aghazadeh, Yadollah Yaghoobzadeh, and Mohammad Taher Pilehvar. DecompX: Explaining transformers decisions by propagating token decomposition. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 2649–2664. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.acl-long.149. URL <https://aclanthology.org/2023.acl-long.149/>.
- Wenbo Pan, Zhichao Liu, Xianlong Wang, Haining Yu, and Xiaohua Jia. Towards long-horizon interpretability: Efficient and faithful multi-token attribution for reasoning LLMs. *arXiv preprint arXiv:2602.01914*, 2026. URL <https://arxiv.org/abs/2602.01914>.
- Hugo Pitorro and Marcos Vinicius Treviso. LaTIM: Measuring latent token-to-token interactions in Mamba models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 24478–24493. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.1194. URL <https://aclanthology.org/2025.acl-long.1194/>.
- Farnoush Rezaei Jafari, Grégoire Montavon, Klaus-Robert Müller, and Oliver Eberle. MambaLRP: Explaining selective state space sequence models. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://papers.neurips.cc/paper_files/paper/2024/hash/d6d0e41e0bled38c76dl3c9e417a8f1f-Abstract-Conference.html.
- Avanti Shrikumar, Peyton Greenside, and Anshul Kundaje. Learning important features through propagating activation differences. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pp. 3145–3153. PMLR, 2017. URL <https://proceedings.mlr.press/v70/shrikumar17a.html>.
- Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pp. 3319–3328. PMLR, 2017. URL <https://proceedings.mlr.press/v70/sundararajan17a.html>.
- Richard S. Sutton, David A. McAllester, Satinder P. Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, volume 12, pp. 1057–1063, 1999. URL <https://proceedings.neurips.cc/paper/1999/hash/464d828b85b0bed98e80ade0a5c43b0f-Abstract.html>.
- Songlin Yang and Yu Zhang. FLA: A triton-based library for hardware-efficient implementations of linear attention mechanism. Software repository, January 2024. URL <https://github.com/fla-org/flash-linear-attention>.
- Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2412.06464>.

A FORMAL RULES AND DERIVATIONS

A.1 FINITE PROPAGATION AND CONTRIBUTION CONSERVATION

For an operation $v = f(u_1, \dots, u_k)$, DELTATRACE constructs linear maps D_j from the two executions such that

$$\Delta v = \sum_{j=1}^k D_j \Delta u_j. \quad (6)$$

An upstream coefficient m_v propagates to input j as $D_j^\top m_v$. Coefficients from multiple consumers add at their shared input. Starting with coefficient 1 at the scalar target, the reverse traversal produces the source coefficients in Equation 2. For a scalar nonlinearity f , the local coefficient is the divided difference $(f(u_1) - f(u_0))/(u_1 - u_0)$, with its derivative as the coincident-input limit. Linear transformations retain their weights, and addition sends the coefficient to each summand.

Theorem 1 (Contribution conservation). *For an acyclic computation over real-valued operations satisfying Equation 6, reverse propagation from a scalar target yields the conservation identity in Equation 2.*

Proof. An acyclic computation can be expanded into elementary operations with explicit branches. At an operation $v = f(u_1, \dots, u_k)$, Equation 6 gives

$$\langle m_v, \Delta v \rangle = \sum_j \langle D_j^\top m_v, \Delta u_j \rangle. \quad (7)$$

Replacing a node’s contribution by the contributions of its parents therefore preserves the total on a cut through the graph. At shared parents, coefficients from different consumers sum. Repeated substitution reaches the input cut and yields Equation 2, since the coefficient of the scalar target is 1. Constant inputs have zero difference and contribute zero to this sum. $\square$

A.2 FINITE ATTENTION NORMALIZATION

For an attention row $p_b = \text{softmax}(z_b)$, $b \in \{0, 1\}$, the logarithmic mean and its sum are

$$\ell_i = \frac{p_{1,i} - p_{0,i}}{\log p_{1,i} - \log p_{0,i}}, \quad \tau = \sum_i \ell_i, \quad (8)$$

with $\ell_i = p_{0,i}$ at coincident probabilities. All quantities are evaluated on the common unmasked support.

Proposition 1 (Finite attention normalization). *For finite logits on a common support,*

$$\Delta p = \left(\text{diag}(\ell) - \frac{\ell \ell^\top}{\tau} \right) \Delta z. \quad (9)$$

Proof. The common support contains positive probabilities. For endpoint b , $\log p_b = z_b - a_b \mathbf{1}$, where $a_b = \log \sum_i \exp(z_{b,i})$. The definition of the logarithmic mean gives

$$\Delta p = \ell \odot (\Delta z - \Delta a \mathbf{1}). \quad (10)$$

Probability normalization implies $\mathbf{1}^\top \Delta p = 0$, so

$$\Delta a = \frac{\ell^\top \Delta z}{\tau}. \quad (11)$$

Substitution proves Equation 9. For a target coordinate y ,

$$\Delta \log p_y = \left(e_y - \frac{\ell}{\tau} \right)^\top \Delta z. \quad (12)$$

The coincident-probability definition follows from the continuous limit of the logarithmic mean. The score operator is symmetric positive semidefinite. For $w_i = \ell_i/\tau$,

$$u^\top \left(\text{diag}(\ell) - \frac{\ell\ell^\top}{\tau} \right) u = \tau \left(\sum_i w_i u_i^2 - \left(\sum_i w_i u_i \right)^2 \right) \geq 0. \quad (13)$$

This is the weighted variance of u multiplied by τ .

□

The operator is symmetric. For an incoming coefficient u_i at probability p_i , the coefficient at its query-key score z_i is

$$m_{z,i} = \ell_i(u_i - \bar{u}_\ell), \quad \bar{u}_\ell = \frac{\sum_j \ell_j u_j}{\sum_j \ell_j}. \quad (14)$$

The response coefficient for a log-softmax target is $m_z = e_y - \ell/\tau$, where e_y is the one-hot vector selecting the target vocabulary coordinate.

A.3 FINITE OPERATOR RULES

For attention output $Y = PV$, Equation 3 gives the coefficients $m_V = \bar{P}^\top U$ and $m_P = U\bar{V}^\top$ for an upstream coefficient U .

Query-key scores use the symmetric product identity,

$$\Delta(QK^\top) = \Delta Q \bar{K}^\top + \bar{Q} \Delta K^\top, \quad \bar{Q} = (Q_0 + Q_1)/2, \quad (15)$$

with $\bar{K}$ defined in the same way. Table 6 lists the finite decompositions by operator. Normalization, rotary position transformations, and grouped heads follow their respective model definitions.

Table 6: Finite propagation rules by operator. Bars denote endpoint averages. The gated-memory recurrence is given in Appendix A.4.

Operation	Change decomposition
Linear and residual	$\Delta(Wx + b) = W\Delta x; \Delta(a + b) = \Delta a + \Delta b$
Attention values	$\Delta(PV) = \bar{P}\Delta V + \Delta P\bar{V}$
Attention scores	$\Delta(QK^\top) = \Delta Q \bar{K}^\top + \bar{Q} \Delta K^\top$
SwiGLU	$\Delta(us) = \bar{s} \odot \Delta u + \bar{u} \odot \Delta s, s = \text{SiLU}(g)$
GDN output gating	$\Delta(n \odot s) = \bar{s} \odot \Delta n + \bar{n} \odot \Delta s$
RMS normalization	Analytic finite rule for $wx/\sqrt{\text{mean}(x^2) + \epsilon}$

A.4 GATED-MEMORY RECURRENCE

With column vectors and a key-by-value state $S_t \in \mathbb{R}^{d_k \times d_v}$, its per-token computation is

$$\begin{aligned} T_t &= \alpha_t S_{t-1}, & r_t &= T_t^\top k_t, \\ c_t &= v_t - r_t, & u_t &= \beta_t c_t, \\ S_t &= T_t + k_t u_t^\top, & o_t &= S_t^\top q_t. \end{aligned} \quad (16)$$

The query includes the model’s fixed attention scale. The state T_t contains retained evidence, r_t reads existing content, and u_t writes the update. One endpoint ordering constructs a finite map D_{GDN}^{01} for the complete memory recurrence through

$$\begin{aligned} \Delta T_t &= \alpha_{t,1} \Delta S_{t-1} + S_{t-1,0} \Delta \alpha_t, \\ \Delta r_t &= (\Delta T_t)^\top k_{t,1} + T_{t,0}^\top \Delta k_t, \\ \Delta u_t &= \beta_{t,1} (\Delta v_t - \Delta r_t) + c_{t,0} \Delta \beta_t, \\ \Delta S_t &= \Delta T_t + k_{t,1} \Delta u_t^\top + \Delta k_t u_{t,0}^\top, \\ \Delta o_t &= (\Delta S_t)^\top q_{t,1} + S_{t,0}^\top \Delta q_t. \end{aligned} \quad (17)$$

Swapping the reference and original activations used as coefficients gives D_{GDN}^{10} . Within each GDN layer, we average these memory-recurrence maps and propagate the readout coefficients as

$$\bar{D}_{\text{GDN}} = \frac{D_{\text{GDN}}^{01} + D_{\text{GDN}}^{10}}{2}, \quad m_{\text{in}} = \bar{D}_{\text{GDN}}^\top m_o. \quad (18)$$

Here m_o collects the coefficients of all readouts o_t in the layer, and m_{in} collects those returned to the recurrence inputs. Equivalently, apply each transpose map to the same m_o and average the returned coefficients. Both maps express the same finite output difference, so their mean preserves it. The recurrence assigns contributions to values, keys, queries, and both gates, carrying credit through stored evidence to earlier sources. Scalar gate nonlinearities and query/key normalization use the surrounding network’s finite rules. Output gating uses the symmetric product in Table 6, with n_t the normalized readout and s_t its activated output-gate vector.

A.5 PRODUCT AND NORMALIZATION IDENTITIES

For a bilinear map B , both

$$\Delta B(a, b) = B(\Delta a, \bar{b}) + B(\bar{a}, \Delta b), \quad (19)$$

$$\Delta B(a, b) = B(\Delta a, b_1) + B(a_0, \Delta b) \quad (20)$$

follow by expansion. Equation 3 and the SwiGLU rule use the first, symmetric identity. The second identity, with the displayed argument ordering, gives the products in Equation 17. Its swapped ordering gives the other layer map averaged in Equation 18. Subtraction and state addition are linear.

For RMS normalization $f(x) = w \odot x/r(x)$, $r(x) = \sqrt{\|x\|^2/d + \epsilon}$, the symmetric product rule gives an analytic coefficient. With $\bar{x} = (x_0 + x_1)/2$, $r_b = r(x_b)$, and $v = w \odot m$, its transpose action is

$$D_f^\top m = \frac{r_0^{-1} + r_1^{-1}}{2} v - \frac{2\bar{x} \langle \bar{x}, v \rangle}{d r_0 r_1 (r_0 + r_1)}. \quad (21)$$

The identity follows from $\Delta r = (2 \langle \bar{x}, \Delta x \rangle / d) / (r_0 + r_1)$ and $\Delta(r^{-1}) = -\Delta r / (r_0 r_1)$. The same construction covers fixed learned scale parameters and the query/key normalization used in the memory path.

B IMPLEMENTATION AND STORAGE ACCOUNTING

Attention. Native endpoint queries, keys, values, and log-normalizers determine the probabilities within each FlashAttention tile. The first sweep accumulates the logarithmic-mean normalizers and weighted centers in Equation 14. Subsequent sweeps accumulate query, key, and value coefficients. Endpoint averages reside in linear-size buffers or are computed inside the tiles.

Memory. The implementation processes 64-token chunks using FLA’s native chunk-state adjoints and batched contractions with saved endpoint states. An associative affine scan computes the finite decay contribution. Convolution and surrounding projections use the installed model primitives and their linear transpose actions.

Storage. For sequence length T , head dimension d , and tile or chunk width C , attention vectors and row statistics occupy $O(Td)$ storage per head. Pairwise products reside in bounded tiles. Dense attention arithmetic is $O(T^2 d)$. Chunked memory occupies $O(Td + TC + (T/C)d^2)$ auxiliary storage per head for token coefficients, within-chunk products, and boundary states. Both storage expressions are linear in T at fixed d and C . Layer replay supplies intermediate activations, with checkpoint storage accounted for separately.

C EVALUATION SPECIFICATION

Qwen3-8B uses the released prompts, stored responses, eligible source positions, and evidence annotations. Qwen3.5 NIAH uses new key/value assignments on fixed templates. Table 5 specifies

the Qwen3.5 examples for each task. An eligible position is an input position included in the task’s attribution and deletion domain. For NIAH, this domain follows the released token selection. VT and HotpotQA recovery select from the passage body.

Deletion scores. For a signed source vector A , DT ranks tokens by A_i for RISE and by $\max(A_i, 0)$ for MAS. The evaluator records 21 response scores across 20 deletion steps. Each score sums log-probabilities over the complete fixed response plus EOS. Full-input and fully-deleted scores define the normalization endpoints. The normalized values are clipped to $[0, 1]$ and made nonincreasing by a cumulative minimum. The case-study curves use the positive ranking and share these endpoints between DT and FT for each model and example.

Baseline comparisons. VT and HotpotQA recovery compare all seven methods on the same 448 inputs. The paired development comparison uses the same inputs and stored responses for DT and FT on each model, with one FT recursive step for faithfulness and three for recovery.

Recovery targets and budgets. NIAH selects the top 10% of eligible input tokens ranked by positive-part attribution scores. VT scores the cached answer tokens and ranks positive body-token scores. Its H2/H4/H6/H10 budgets are 10%/10%/20%/30%. HotpotQA scores the full response, ranks native body sentences by the sum of their positive token contributions, and selects the longest ranked sentence prefix within 30% of all body tokens. NIAH/VT report the fraction of annotated evidence tokens recovered. HotpotQA reports Full-support Recall@30%. Each example scores one if every official supporting fact is selected and zero otherwise. The reported value is the mean across examples.

Table 7: Paired development results on the first eight MQ-Q2 and eight MoreHopQA examples per model. These 32 model–example pairs form a separate development set. FT uses one recursive hop for RISE/MAS and three for recovery. RISE is signed, while MAS and recovery use positive scores.

Model	Task	Method	RISE ↓	MAS ↓	Recovery@10% ↑
Qwen3-8B	MQ-Q2	FlashTrace	0.0685	0.1547	56.45
		DELTA TRACE	0.0574	0.0806	77.52
	MoreHopQA	FlashTrace	0.1111	0.1969	—
		DELTA TRACE	0.0585	0.1173	—
Qwen3.5-9B	MQ-Q2	FlashTrace	0.0942	0.2485	63.92
		DELTA TRACE	0.1115	0.1518	73.35
	MoreHopQA	FlashTrace	0.2356	0.2797	—
		DELTA TRACE	0.1688	0.2426	—

D IMPLEMENTATION COST MEASUREMENTS

Each timing comparison uses the same inputs and responses for its alternatives. Complete attribution time covers the two forward executions, the reverse pass, and returning the source scores. Peak allocated device memory includes model weights. Sequence length counts both input and response tokens.

Paired Qwen3 cost. Every method has one warm call followed by three measured calls in rotated order on each of 16 development examples. Per-example medians are aggregated by task. The maximum allocated memory includes model weights. Three-hop FlashTrace takes 18.27 seconds for the same 16 examples.

Tiling on longer released examples. Figure 2 compares dense and tiled finite propagation on the 1,241-token retrieval control, the longest released HotpotQA context (3,470 total tokens), and the longest MATH rollout (3,762 total tokens). At 3,470 tokens, tiling reduces complete attribution time from 5.32 to 3.02 seconds and peak allocation from 32.66 to 22.54 GB. At 3,762 tokens, the corresponding values are 6.12 to 3.40 seconds and 37.18 to 30.06 GB.

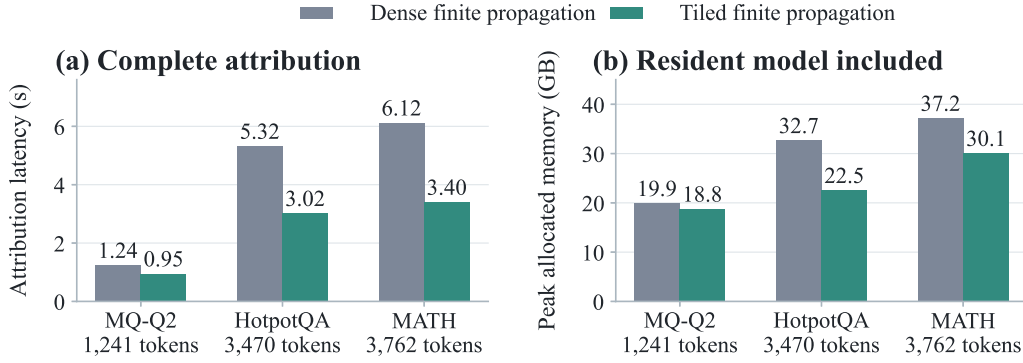


Figure 2: **Dense and tiled finite propagation on three released Qwen3-8B examples.** Complete warm attribution latency is the median of repeated calls. Memory is peak allocated device memory, including resident weights. Each input keeps its original prompt and full stored response. The three groups are distinct tasks and response lengths. Bars show the measured examples.

Batching and replay retention. The Qwen3.5 comparison processes the same 16 examples serially and in batches of two. With GPU checkpoints and batching, time decreases from 16.68 to 12.33 seconds. Retaining activations for reuse during the reverse pass further reduces time by 5.80% on Qwen3 and 5.15% on Qwen3.5 (Table 8).

Table 8: Effect of retaining replay activations on complete attribution cost for 16 examples. Each row compares execution without and with retained replay activations. Warm time is the mean complete pass over 16 examples across two interleaved rounds. Peak memory gives before/after allocations, including resident weights.

Model	Sample batch	Before (s)	Retained (s)	Reduction	Peak GB (before/after)
Qwen3-8B	1	10.360	9.759	5.80%	18.76 / 18.67
Qwen3.5-9B	2	11.480	10.889	5.15%	23.63 / 23.63

E DETAILED SIGNED EVIDENCE VIEWS

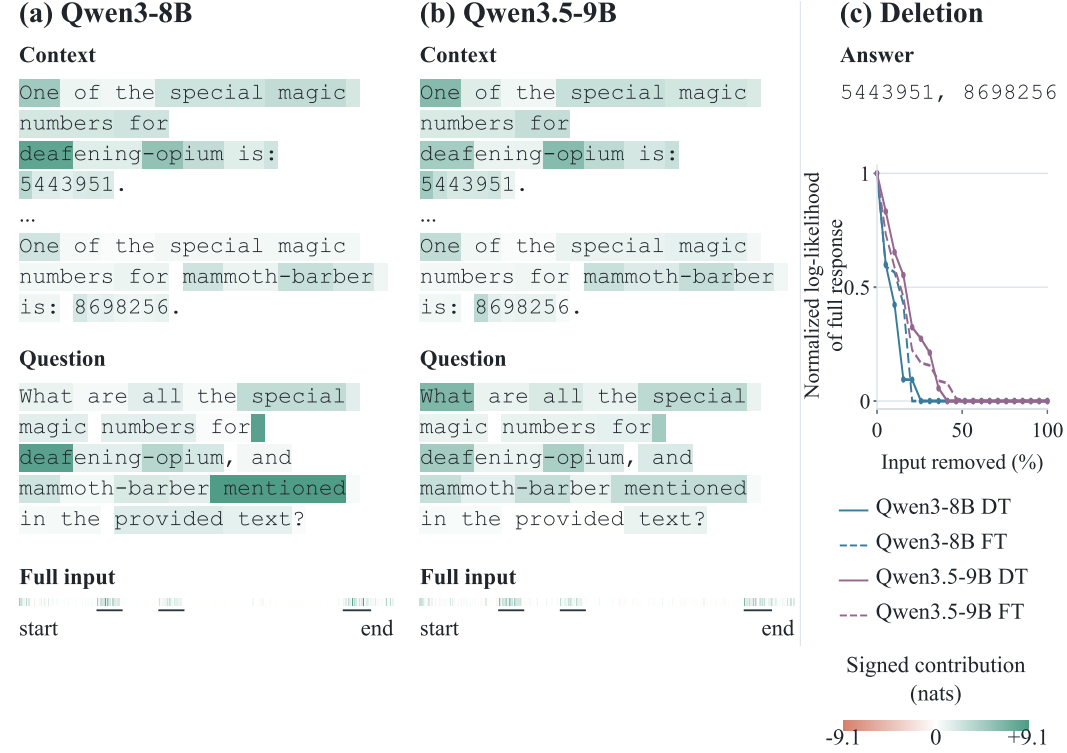


Figure 3: **Signed evidence for retrieving two numbers.** (a,b) The same `niah_mq-q2` input on Qwen3-8B and Qwen3.5-9B. Teal and coral show positive and negative DT contributions in nats. Marks locate excerpts. (c) Each model’s DT (solid) and one-hop FT (dashed) curves show the normalized log-likelihood of the entire fixed response plus EOS under 20-step deletion. Color identifies the model. The released normalization uses that model’s full-input and fully-deleted scores, clipping and a cumulative minimum. DT deletion uses positive ranking. Answer summarizes the full stored response.

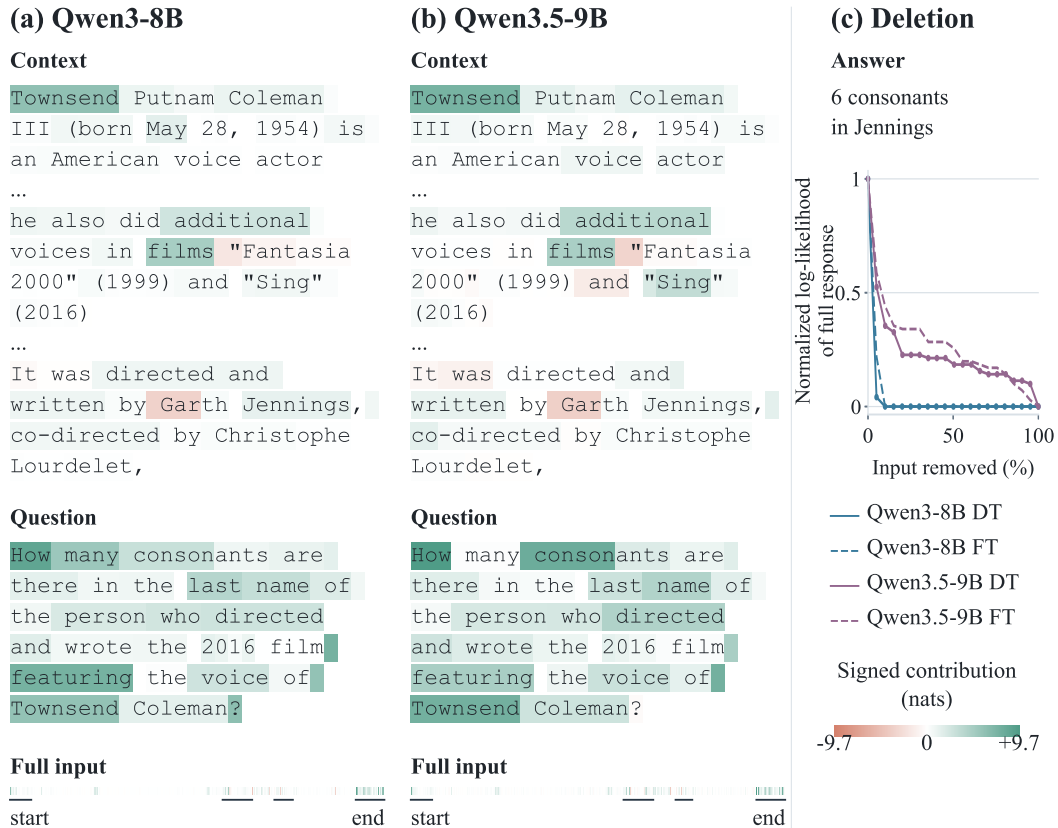


Figure 4: **Signed evidence across a multi-hop question.** (a,b) The MoreHopQA input links Townsend Coleman to *Sing*, Garth Jennings, and his surname’s consonants. (c) Normalized log-likelihood of the entire fixed response plus EOS, comparing each model’s DT with its own one-hop FT baseline. Model colors, method line styles, and per-model normalization follow Figure 3. Both heatmaps share the signed scale within this example.

F COUNTERFACTUAL CREDIT AND POLICY LEARNING

The response attribution studied here suggests a broader research direction based on propagating finite reward differences to actions that contribute to them. The connection to reinforcement learning is particularly direct when counterfactual returns are averaged exactly. The following identity expresses the familiar action-independent-baseline principle (Sutton et al., 1999) in counterfactual form. COMA uses the related construction to assign credit by averaging one agent’s action while fixing the others (Foerster et al., 2018).

Let h be a decision history and π a fixed continuation policy. Write $G^\pi(h, a; \xi)$ for the integrable return obtained by setting the current action to a and subsequently following π , with ξ supplying environment and policy randomness. Define

$$Q^\pi(h, a) = \mathbb{E}_\xi[G^\pi(h, a; \xi)], \quad V^\pi(h) = \mathbb{E}_{a' \sim \pi(\cdot|h)}[Q^\pi(h, a')]. \quad (22)$$

The reference action a' is sampled independently of the selected action a , conditional on h . The two rollouts may share randomness, with each retaining its correct marginal distribution under the corresponding action intervention.

Proposition 2 (Exactly averaged counterfactual credit). *Under these definitions, the policy-averaged counterfactual credit satisfies*

$$C^\pi(h, a) = \mathbb{E}_{a', \xi}[G^\pi(h, a; \xi) - G^\pi(h, a'; \xi)] = Q^\pi(h, a) - V^\pi(h). \quad (23)$$

For a differentiable policy π_θ with positive probabilities over a fixed finite action set, the score-function learning signals obey

$$\mathbb{E}_{a \sim \pi_\theta}[\nabla_\theta \log \pi_\theta(a | h) C^{\pi_\theta}(h, a)] = \mathbb{E}_{a \sim \pi_\theta}[\nabla_\theta \log \pi_\theta(a | h) Q^{\pi_\theta}(h, a)]. \quad (24)$$

Proof. Linearity of expectation gives Equation 23. The baseline term in Equation 24 is

$$V^{\pi_\theta}(h) \sum_a \pi_\theta(a | h) \nabla_\theta \log \pi_\theta(a | h) = V^{\pi_\theta}(h) \nabla_\theta \sum_a \pi_\theta(a | h) = 0. \quad (25)$$

Subtracting this term proves the result. Credit and action value serve as weights in the score-function update. $\square$

This equality provides a concrete target for extending finite attribution to learning. The averaged action credit of a reward-level construction should recover the advantage. The resulting research questions concern how computational or environmental structure can make this credit efficient to estimate, how reference choices and coupled rollouts affect its variance, and how the estimate behaves when a trajectory is extended by steps irrelevant to the reward. These questions connect signed attribution to the quality and temporal stability of a learning signal.